

Betsy -- Autonomous Procurement Agent

Betsy is an AI agent that monitors inventory, detects procurement problems, and either acts on them autonomously or escalates them to a human for approval. Built as a school research project exploring what it takes to make an AI system that a non-technical operations manager can actually trust.

What it does

Betsy runs a procurement pipeline every 30 minutes (or on demand) and handles four failure modes:

- Stockout risk -- stock below reorder point, agent generates a PO to the best-scoring supplier
- Price spike -- unit price significantly above historical average, flags for human review
- Duplicate invoice -- same supplier, same amount, same period -- flags as potential fraud
- Supplier unavailable -- finds the next best alternative automatically

Any decision above the \$5,000 autonomous spending limit is held in an approval queue for a human to approve or decline. Everything below it executes automatically and logs to the audit trail.

Stack

```
Layer | Tech
-----
Agent framework | LangGraph (Pipeline + Orchestra patterns)
-----
LLM | Ollama -- llama3.1:8b (local)
-----
Backend | FastAPI + uvicorn
-----
Persistence | SQLite (built-in sqlite3)
-----
Scheduling | APScheduler BackgroundScheduler
-----
Frontend | Vanilla JS, no framework
-----
```

Setup

Prerequisites: Python 3.11+, Ollama running locally with llama3.1:8b pulled.

```
# install dependencies
pip install -r requirements.txt

# pull the model (if not already done)
ollama pull llama3.1:8b

# start the server
python run_server.py
```

Open <http://localhost:8000/betsy> for the agent UI (Jenny's view) or <http://localhost:8000> for the full dev dashboard.

Optional flags

<code>python run_server.py --port 8080</code>	<code># different port</code>
<code>python run_server.py --interval 2</code>	<code># auto-run every 2 minutes</code>
<code>python run_server.py --no-reload</code>	<code># disable hot reload</code>

Project structure

<code>betsy-ai-agent/</code>	
<code>??? server/</code>	FastAPI app, routers, SQLite persistence
<code>? ??? main.py</code>	App entry point, lifespan, scheduler setup
<code>? ??? db.py</code>	SQLite write-through layer
<code>? ??? state.py</code>	In-memory state, scenario injection
<code>? ??? scheduler_instance.py</code>	APScheduler singleton
<code>? ??? routers/</code>	inventory, suppliers, orders, approvals, stats...
<code>??? pipeline/</code>	LangGraph pipeline (sequential)
<code>? ??? graph.py</code>	6-node workflow definition
<code>? ??? nodes/</code>	ingest -> detect -> evaluate -> decide -> act -> audit
<code>??? orchestra/</code>	LangGraph orchestra (parallel multi-agent)
<code>??? dashboard/</code>	
<code>? ??? betsy.html</code>	Agent UI -- for the operations manager
<code>? ??? index.html</code>	Dev dashboard -- full data view
<code>??? mock_data/</code>	12 SKUs, 6 suppliers, invoices, purchase orders
<code>??? scenarios/</code>	4 injectable test scenarios
<code>??? tests/</code>	
<code>? ??? test_ema_learning.py</code>	EMA formula verification
<code>? ??? test_long_term_learning.py</code>	Integration test -- 2 LLM runs, 8 delivery rounds
<code>??? docs/</code>	Architecture docs, API spec, test reports
<code>??? decision_logs/</code>	7 decision logs documenting every major build choice
<code>??? run_server.py</code>	Startup script

The two UIs

`betsy.html` -- designed for a non-technical operations manager. Plain English narrative, confidence shown as dots not decimals, approval cards that explain why Betsy stopped before asking for a decision. No jargon.

`index.html` -- full dev view. Raw inventory table, supplier scoreboard with live EMA scores, agent log, purchase order history.

Both surfaces read from the same API and update every 5 seconds.

Learning

Betsy gets smarter from delivery history. Every time a PO is marked delivered, the supplier's reliability score updates via an exponential moving average:

```
new_score = 0.2 × delivery_performance + 0.8 × old_score
performance = max(0.0, 1.0 - lateness_days × 0.1)
```

A supplier who delivers 8 days late repeatedly drops from a 0.92 score to 0.44 over 5 deliveries. That changed score changes who Betsy orders from next time -- demonstrated in

tests/test_long_term_learning.py.

Decision logs

Seven decision logs in decision_logs/ document every major choice made during the build -- framework selection, architecture tradeoffs, UI design rationale, the HITL approval flow, persistence strategy, and the learning mechanism. Each log follows the DOT framework research format used in the ICT bachelor programme.

```
Log | What it covers
-----
DL-01 | Building the mock environment
-----
DL-02 | Pipeline vs Orchestra architecture
-----
DL-03 | UI design for non-technical users
-----
DL-04 | First real run and model selection
-----
DL-05 | HITL approval queue end-to-end
-----
DL-06 | SQLite persistence, EMA learning, APScheduler
-----
DL-07 | Proving that score learning changes decisions
-----
```

Running tests

```
# start the server first
python run_server.py

# EMA formula verification (fast)
python tests/test_ema_learning.py

# Long-term learning integration test (3-5 min, requires LLM)
python tests/test_long_term_learning.py
```